Green Computing Strategies for Healthcare Predictive Models

Güldeniz Güzelay, Sude Aslan
University of Milano Bicocca/Data Science, Milano, İtaly
g.guezelay@campus.unimib.it -946901
s.aslan1@campus.unimib.it - 946809
CodeBerg: https://codeberg.org/Unimib_Green_Computing_Project/GreenEHR

ABSTRACT

The project was prepared for the Green Computing course in the Master's Degree Program in Data Science at the University of Milano-Bicocca. During the project, comprehensive data preprocessing and feature engineering techniques were applied, including data cleaning, missing value handling, duplicate removal, categorical encoding, feature scaling, and domain-specific feature engineering. Potential data leakage and redundant features were identified and removed to ensure reliable model evaluation. Clinically meaningful variables were also generated to improve predictive performance while maintaining interpretability.

Three machine learning algorithms, Logistic Regression, Random Forest, and XGBoost, were trained and evaluated using a unified preprocessing pipeline. Model performance was assessed using the Matthews Correlation Coefficient (MCC). To obtain robust results, each experiment was repeated over 100 iterations using different train-test splits.

In addition to predictive performance, the environmental impact of model execution was measured using CodeCarbon. Energy consumption, CO_2 emissions, and execution time were recorded for each model and dataset combination. The results provide insights into the relationship between model accuracy and computational sustainability, highlighting the importance of considering environmental metrics when developing machine learning solutions.

INTRODUCTION

In this project, we have 5 different datasets.Each dataset is related to a different disease and each of them has its own output variable.

For each dataset, we have common features like age,sex and the output variable, as well as specific features related to that particular disease. We read each dataset from Google Drive by using Google Colab. As this project is a teamwork effort, these tools enable us to work together effectively.
During the project process, the data will first be explored through exploratory data analysis (EDA). For predicting output variables, we used machine learning tools and measured this process with MCC values and specific Codecarbon parameters. In the second part, we wrote a green version of our model with energy-saving strategies aiming to reach the same MCC value with lower CodeCarbon parameters. Finally, the second version of the model is converted to the R language to see the impact of the language difference on the CodeCarbon parameters.

1.FIRST MODEL WITHOUT GREEN COMPUTING PERSPECTIVE

1.1.Data Preparation and Cleaning

After collecting five Electronic Health Record(EHR) datasets, each dataset was analyzed individually to understand its structure, feature types and meaning, target variable and potential insight.

Datasets:

- Brain_tumor = 173 x 31
- Cancer = 998 x 32
- Heart_failure = 425 x 15
- Sepsis = 1257 x 16
- Neuroblastoma = 169 x 13

Before model development, a data cleaning process was applied to all datasets.
Firstly, all datasets' column names started the standardization process. It includes removal of unnecessary spaces and formatting inconsistencies. Then, the Brain_tumor data set has some duplicate values and during the project all datasets detect it. After all, all of the column types and missing values are checked. Also, feature converted boolean(0/1) format.

1.2.Data Leakage Detection and Removal

For all datasets, the target variable was originally located as the last column. However, during preprocessing steps like feature engineering and one hot encoding, the position of the target variable could be changed. That's why, in the beginning of the code, a predefined list of target variables was created and matched with datasets name to ensure that the correct target variable was consistently identified throughout the analysis.

Special attention was given to features that could introduce data leakage. For example, "time from heart failure to death" feature had future information unavailable at prediction time and they were remove

1.3. Feature Engineering

After searching specific domain knowledge for all data sets, feature engineering was prepared separately for each of them and used medical knowledge into the modeling process. During the feature engineering, new clinically meaningful variables were created by combining existing features and deriving additional indicators associated with disease severity, patient risk, and treatment characteristics.
Example of features during the process:

Neuroblastoma Dataset

- Age Risk Group
- Disease Severity Score
- Treatment Intensity Score

Pediatric Brain Tumor Dataset

- Tumor Burden Score
- Age Category
- Treatment Complexity Score

Colorectal Cancer Dataset

- Combined Tumor Marker Indicators
- Risk Stratification Variables
- Age-Based Risk Features

Sepsis/SIRS Dataset

- İnflammation Ratios
- Organ Dysfunction Indicators
- Severity Scores

Depression with Heart Failure Dataset

- Comorbidity Scores
- Depression Severity Indicators
- Heart Failure Risk Factors

1.4. Data Preprocessing Pipeline

The features were divided into numerical and categorical variables, and appropriate preprocessing techniques were applied to each group.

For numerical features, missing values were handled using median imputation. For Logistic Regression models, feature scaling is also applied when required with the Min Max Scaler function. Then it is followed by one-hot encoding to convert categorical variables into a numerical format suitable for machine learning algorithms.

A unified preprocessing pipeline was developed and applied across all datasets to ensure consistency, reproducibility, and fair comparison of model performance throughout the experiments.

1.5. Machine Learning Models

Three machine learning algorithms which were Logistic Regression, Random Forest and Extreme Gradient Boosting were selected and evaluated in this study.

1. *Logistic Regression*: It is used as a lightweight and interpretable baseline model. Its simplicity and low computational requirements make it suitable for comparison with more complex approaches.

2. *Random Forest*: It was employed to capture nonlinear relationships and feature interactions through an ensemble of decision trees.
3. *Extreme Gradient Boosting (XGBoost):* it was included as a high-performance ensemble learning method known for its strong predictive capabilities and efficient handling of structured data.

To ensure a fair comparison, all models were trained using the same preprocessing pipeline and evaluation procedure across all datasets.

1.6. Performance Metrics

The primary evaluation metric used in this study was the Matthews Correlation Coefficient (MCC). MCC was selected because it is robust to class imbalance, considers all components of the confusion matrix, and provides a balanced measure of classification performance.

Dataset	Model	Target Column	mean_ mcc
neuro	RF	outcome	46,56%
braint	RF	Status OS	78,17%
cancer	RF	death	21,79%
sepsis	RF	Mortality	52,49%
depres	RF	Death (1=Yes, 0=No)	36,11%
neuro	LR	outcome	49,49%
braint	LR	Status OS	69,26%
cancer	LR	death	24,53%
sepsis	LR	Mortality	57,16%
depres	LR	Death (1=Yes, 0=No)	38,19%
neuro	XGBOOST	outcome	52,83%
braint	XGBOOST	Status OS	82,82%
cancer	XGBOOST	death	25,68%
sepsis	XGBOOST	Mortality	53,63%
depres	XGBOOST	Death (1=Yes, 0=No)	42,20%

Table 1 : Model-MCC results

The environmental impact of model execution was measured using CodeCarbon. For each model and dataset combination, energy consumption, CO_2 emissions, and execution time were recorded.

Model	Energy_ Wh	Emission_g _CO2eq	Duration_Seconds_ Codecarbon
RF	2,92	0,4052	200,259
LR	0,21	0,0296	14,6387
XGBOOST	0,57	0,08039	39,7331

Table 2 : Model-CodeCarbon Results

These measurements enabled a comparison of predictive performance and environmental sustainability across different machine learning models.

2.MODEL WITH GREEN COMPUTING PERSPECTIVE

In our model it applied green strategies to reach the same MCC but more optimized and energy saver version of our model.
Applied strategies:

2.1. Pruning

Feature Pruning: Based on the feature importance output, we dropped features with a score of less than **0.3%**. In this way, we decreased the matrix dimensions to achieve lower energy consumption.

Random Forest Pruning:We prevented the construction of overly large trees by tuning specific parameters. Setting *max_depth=6*, *min_samples_leaf=3, min_samples_split=6* and *ccc_alpha=1e-4* constrained the model to build a lighter random forest.

XGBoost Pruning: Similar to Random Forest it used constant parameters to prevent overfitting and massive model construction. We set *max_depth=2 ,min_child_weight=3, gamma=0.1 and* applied L1/L2 regulation *(reg_alpha=0.1, reg_lambda=2.0)*.

2.2 Quantization

Using the downcast_numeric_features_to_float32 function, all numeric features were downcast from float64 to float32.This is highly beneficial for our pipeline since most of our datasets primarily consist of float numbers. This process reduces memory bandwidth and optimizes CPU cache management,

which ultimately leads to lower Watt consumption and less hardware heating.

2.3 Computation-Aware Training

Iteration numbers (number of estimators) were optimized and downscaled for both Random Forest and XGBoost. Additionally, we utilized histogram-based XGBoost (tree_method="hist") to bin continuous numeric values into discrete intervals. This significantly accelerated the tree-building pace while reducing CPU cycle counts. For Logistic Regression, we selected the lightweight, memory-friendly binary solver solver="liblinear" and relaxed the convergence tolerance to tol=1e-3, thereby minimizing the processor's matrix multiplication loops.

2.4 Hardware-Aware Efficiency & Operational Tracking

Thread Limit and Single Core: By setting n_jobs=1 in the models, we prevented the operating system from performing frequent context-switching across multiple CPU cores. This eliminated thread-management overhead and ensured a more stable, lower-overhead execution.
Emissions Tracking (CodeCarbon): With the integration of the *EmissionsTracker* library, power consumption (Watts) was monitored at the CPU and RAM levels, allowing us to calculate and record the net energy consumption (Wh) and CO_2 emissions (g).

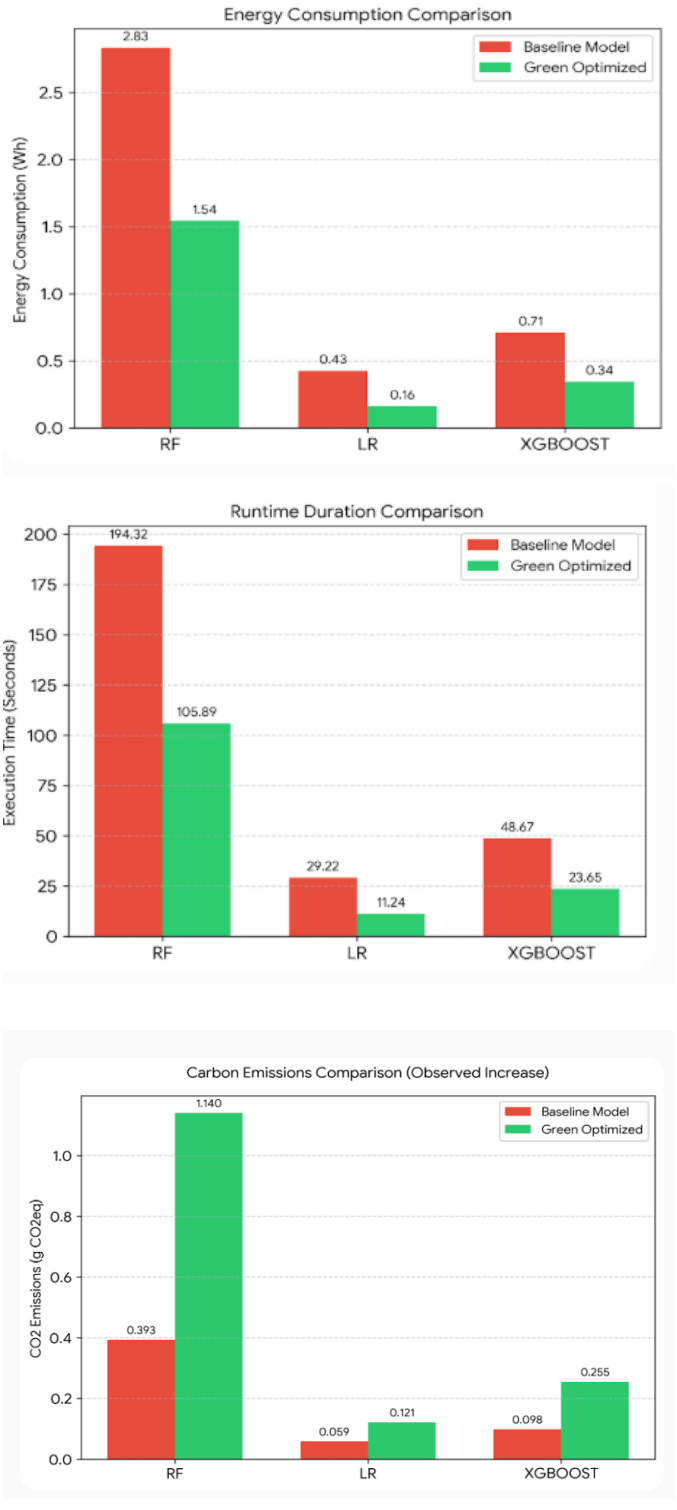
2.5. Reduced Training Overhead & Metric Streamlining

Unnecessary calculations and algorithmic overhead are eliminated from the core hold-out loop to streamline execution:

No inside Hyperparameter Search: In the 100 iteration hold-out there are no heavy hyperparameters like *GridSearchCV* and *RandomizedSearchCV* .

Metric Streamlining: In the iteration we removed Accuracy, Precision, Recall, F1 and ROC-AUC calculations. Only added the main performance parameter MCC . Therefore, the model is spent less time for execution and saves energy.

With this green approach, the changes in CodeCarbon parameters are visualized in the graphs below:



Graph 1-2-3 : CodeCarbon Results Comparison

2.6 Conclusion

The green version successfully reduced computational requirements across all models. Total energy consumption decreased from 3.97 Wh to 2.05 Wh (48.4% reduction), while execution time decreased from 272.2 seconds to 140.8 seconds (48.3% reduction). Although the measured CO_2eq values increased due to changes in the estimated carbon intensity of the execution environment, the reduction in energy consumption and runtime demonstrates that the proposed optimization strategies effectively improved computational efficiency.

3.PYTHON AND R LANGUAGE

For the R component of our study, we translated the 'Green' version of our model into the R language to analyze how execution time, energy consumption, and CO_2 emissions vary across platforms.

Due to the different methodologies inherent to Python and R, as well as variations in the training and test splits, the model performance is changed.While XGBoost MCC values slightly increased by 1.20% in R, Random Forest decreased by 4.52% and Logistic Regression dropped by 16.52% when moving from Python to R. However, we observed substantial differences in the CodeCarbon metrics depending on the specific model applied to the data.

When analyzing these carbon footprint parameters across languages, our comparisons were structured directly around the individual machine learning models used.

For Random Forest, R is much faster and uses less energy than python. For time, R using 7.39 seconds on average, python uses 26.99.R uses only 0.57 Wh of energy and creates 0.12 g of . Python uses 1.98 Wh of energy and creates 0.56 g of CO_2.

Python takes 2.96 seconds on average, while R takes 5.07 seconds. Both of them complete the task very quickly, their energy use (0.23 Wh vs 0.25 Wh) and CO_2 emissions (0.06 g vs 0.05 g) are almost the same.

For Xgboost , Python finishes in 5.05 seconds on average, while R takes 6.04 seconds. Python is slightly faster. Even though the times are close, R uses 1.5 times more energy than Python. Python uses 0.37 Wh of energy, while R uses 0.55 Wh. R also produces more CO_2(0.12 g vs Python's 0.11 g).

CodeCarbon Parameter Deviations Across Platforms (Python >R)

Model	ΔMean Elapsed Time	ΔMean Energy Consumption (Wh)	ΔMean CO2 Emissions (g)
RF	-72.62%	-71.21%	-78.57%
LR	71.28%	8.70%	-16.67%
XGBOOST	19.60%	48.65%	9.09%

Table 3: CodeCarbon parameters changing from Python to R

Efficient Summary:

- RF: R is significantly more efficient. It executed 3.6 times faster, drastically reducing both energy draw and carbon footprint.
- LR:Marginal trade-off. Although execution runtime increased in R, energy consumption and emissions remained closely matched due to the model's low mathematical complexity.
- XGBOOST : Python is more efficient. R operated 19.6% slower and required approximately 1.5 times more energy to complete the same operational load.

REFERENCES

[1] CodeCarbon Contributors. (2025). *CodeCarbon: Track and reduce the carbon footprint of your machine learning models*. Python Package Index. https://pypi.org/project/codecarbon/ othenburg. https://www.v-dem.net/documents/70/codebook_v16.pdf